

UNIT IV

Optimization for Training Deep Models

Introduction

- In deep learning, an optimizer is a crucial element that dynamically fine-tune a model's parameters throughout the training process, aiming to minimize a predefined loss function. Its primary role is to minimize the model's error or loss function, enhancing performance. Various optimization algorithms, known as optimizers, employ distinct strategies to converge towards optimal parameter values for improved predictions efficiently.
- These specialized algorithms facilitate the learning process of neural networks by iteratively refining the weights and biases based on the feedback received from the data.
- Each optimizer is equipped with distinct update rules, learning rates, and momentum strategies, all geared towards the overarching goal of discovering and converging upon optimal model parameters, thereby enhancing overall performance.
- Stochastic Gradient Descent, Mini-Batch Gradient Descent, Adagrad, RMSProp, AdaDelta, and Adam

Important Deep Learning Terms

Before proceeding, there are a few terms that you should be familiar with.

- **Epoch** – The number of times the algorithm runs on the whole training dataset.
- **Sample** – A single row of a dataset.
- **Batch** – It denotes the number of samples to be taken to for updating the model parameters.
- **Learning rate** – It is a parameter that provides the model a scale of how much model weights should be updated.
- **Cost Function/Loss Function** – A cost function is used to calculate the cost, which is the difference between the predicted value and the actual value.
- **Weights/ Bias** – The learnable parameters in a model that controls the signal between two neurons.

How Learning Differs from Pure Optimization

- **learning** is **not just** about optimizing a cost function. Instead, learning in deep models has additional challenges beyond pure optimization.
- **Generalization vs. Optimization**
- In traditional optimization, the goal is to find the lowest possible value of the objective function.
- In machine learning, particularly deep learning, the main goal is **generalization**—ensuring the model performs well on unseen data.
- A model that overfits training data may achieve low cost but fail to generalize. Hence, the best optimization result is not always the best learning outcome.

III-Conditioning of Cost Functions

- Deep learning cost functions can have flat regions, sharp ridges, or chaotic landscapes, making pure optimization inefficient.
- Algorithms like Adam and RMSprop help navigate these difficult optimization surfaces.

Role of Regularization

- Regularization techniques (L1/L2 penalties, dropout, data augmentation) modify the optimization process to encourage better generalization.
- This differs from pure optimization, which only seeks to minimize the cost function without considering overfitting.

Optimization Does Not Necessarily Need to Reach Global Minimum

- Unlike convex optimization problems where the global minimum is ideal, deep learning models often work well with **local minima** or even saddle points.
- Empirical studies show that many local minima in deep networks have similar generalization performance.

Noisy and Stochastic Training

- Unlike deterministic optimization problems, deep learning involves **stochasticity** due to random weight initialization, mini-batch training, and data noise.
- Methods like SGD leverage this stochastic nature to help escape saddle points and improve generalization.

Implicit Bias of Optimization Algorithms

- Different optimization algorithms (SGD, Adam, etc.) introduce biases that affect model performance.
- SGD, for example, tends to favor solutions with better generalization properties, even if they are not the absolute minima.

Challenges in Optimization for Training Deep Models

Optimizing deep neural networks is much more complex than optimizing simple functions. Several challenges arise due to the high-dimensional, non-convex nature of deep learning cost functions. Below are some of the key challenges in optimization:

1. Vanishing and Exploding Gradients

- During backpropagation, gradients can become extremely small (vanishing) or excessively large (exploding), making training unstable.
- This typically occurs in deep networks where repeated multiplication of small or large derivatives leads to extreme values.
 - Vanishing gradients slow down learning, especially in earlier layers.
 - Exploding gradients cause weight updates to be too large, leading to instability.

To overcome:

- Use activation functions like **ReLU** instead of sigmoid or tanh.
- Employ **batch normalization** to normalize activations.
- Use **gradient clipping** to prevent extreme values.

Poor Conditioning of the Cost Function

- The optimization landscape may contain regions where gradients change rapidly in some directions and slowly in others.
- Happens when the Hessian matrix (second derivative of the cost function) has a high condition number, leading to slow convergence.
 - Standard gradient descent struggles with poorly conditioned surfaces.
 - Training can take much longer to converge.
- **To overcome**
 - Use **adaptive learning rate algorithms** like RMSprop, Adam, or Adagrad.
 - Apply **batch normalization** to improve conditioning.
 - Scale input features properly to ensure a well-behaved cost function.

Saddle Points and Local Minima

- Unlike convex optimization, where there is one global minimum, deep learning cost functions contain many **local minima** and **saddle points** (where the gradient is zero but it is neither a minimum nor a maximum).
- High-dimensional spaces often have many saddle points, making optimization harder.
 - SGD and other optimizers can get stuck in these regions, slowing down training.
 - Many local minima may perform equally well, but poor local minima can degrade generalization.
- **To overcome**
 - Use **stochastic gradient descent (SGD)** with momentum to help escape saddle points.
 - Adaptive optimization methods like **Adam** can help navigate complex landscapes.
 - Introduce **random weight initialization** to prevent poor local minima trapping.

Choosing the Right Learning Rate

- A learning rate that is too high causes divergence, while a rate that is too low results in slow convergence.
 - Poorly chosen learning rates lead to inefficient training and may cause oscillations.

To overcome

- Use **learning rate schedules** (e.g., step decay, cosine annealing, or cyclical learning rates).
- Implement **adaptive learning rate algorithms** (Adam, RMSprop, Adagrad).
- Perform **hyperparameter tuning** to find an optimal learning rate.

Computational Cost and Hardware Limitations

- Training deep models requires substantial computational resources, making optimization time-consuming.
 - Large-scale datasets and deep models demand high GPU/TPU memory.
 - Optimization algorithms can be inefficient when dealing with large-scale deep networks.
- To overcome:
 - Use **mini-batch training** instead of full-batch updates.
 - Optimize computation using **parallel processing** and **distributed training**.
 - Use **model pruning, quantization, and knowledge distillation** to reduce computational overhead.

Hyperparameter Sensitivity

- Deep learning models depend heavily on hyperparameters (e.g., learning rate, batch size, momentum). Small changes can drastically impact performance.
 - Poorly chosen hyperparameters lead to slow convergence or instability.
- To overcome
 - Use **automated hyperparameter tuning** (grid search, random search, Bayesian optimization).
 - Experiment with different batch sizes and optimizers to find an optimal balance.

Optimizing deep neural networks involves more than just minimizing a function—it requires navigating challenges like vanishing gradients, saddle points, generalization, and computational constraints. Modern deep learning techniques employ various **adaptive learning rates, regularization methods, and specialized optimization algorithms** to address these challenges effectively.

Basic Algorithms

Stochastic Gradient Descent(SGD)

SGD is one of the most widely used algorithms for training deep learning models.

- **Gradient Descent** involves computing the gradient of the cost function with respect to the parameters across the *entire* training set.
- **Stochastic Gradient Descent** accelerates this by computing the gradient based on a **randomly selected mini-batch** of examples. This provides a noisy but efficient estimate of the true gradient.

Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k .

Require: Initial parameter θ

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow +\frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Apply update: $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$

end while

Inputs:

- **Learning rate ϵ_k :**
A small scalar that controls how big a step we take in the direction of the negative gradient.
- **Initial parameter θ :**
The model's starting weights (e.g., in a neural network).

Loop (Repeat until convergence):

1. Mini-batch Sampling:

“Sample a minibatch of m examples from the training set $\{x^{(1)}, \dots, x^{(m)}\}$ with corresponding targets $y^{(i)}$.”

This step selects a small random subset of training data (called a minibatch). Each $x^{(i)}$ is an input, and each $y^{(i)}$ is its corresponding label.

2. Gradient Estimation:

$$\hat{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$$

Here's what's happening:

- $f(x^{(i)}; \theta)$: Model prediction for input $x^{(i)}$
- $L(\cdot)$: Loss function (e.g., cross-entropy, MSE)
- ∇_{θ} : Gradient with respect to model parameters
- The whole expression computes the **average gradient of the loss** over the minibatch

3. Parameter Update:

$$\theta \leftarrow \theta - \epsilon_k \hat{g}$$

- Update parameters by taking a step **opposite** to the gradient (to reduce the loss).
- The size of the step is controlled by the learning rate ϵ_k .

Momentum

- While stochastic gradient descent remains a very popular optimization strategy, learning with it can sometimes be slow. The method of momentum (Polyak 1964 ,) is designed to accelerate learning, especially in the face of high curvature, small but consistent gradients, or noisy gradients. The momentum algorithm accumulates an exponentially decaying moving average of past gradients and continues to move in their direction.
- Momentum improves SGD by helping the optimizer **build velocity** in directions of consistent gradient descent, reducing oscillations.
- **Mechanism:**
- Instead of updating the parameters purely by the current gradient, momentum introduces a **velocity vector** that aggregates past gradients.
- This technique helps traverse ravines(deep narrow steeps) and escape small local minima more effectively.

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α .

Require: Initial parameter θ , initial velocity v .

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{x^{(1)}, \dots, x^{(m)}\}$ with corresponding targets $y^{(i)}$.

 Compute gradient estimate: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$

 Compute velocity update: $v \leftarrow \alpha v - \epsilon g$

 Apply update: $\theta \leftarrow \theta + v$

end while

Inputs:

- **Learning rate ϵ :** Controls the step size of the parameter update.
- **Momentum parameter α :** Controls how much of the past velocity (update direction) is retained.
- **Initial parameter θ :** Model weights at the beginning.
- **Initial velocity v :** Typically initialized as 0 (a vector of the same shape as θ).

Loop (Repeat until stopping criterion is met):

1. Mini-batch Sampling:

Sample m examples $\{x^{(1)}, \dots, x^{(m)}\}$ and their targets $y^{(i)}$

This is just like in standard SGD — take a small random subset of the training data.

2. Gradient Estimate:

$$g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$$

- Calculate the **average gradient** of the loss function over the mini-batch.
- This gradient points in the direction of steepest ascent of loss — we want to go the other way.

3. Velocity Update:

$$v \leftarrow \alpha v - \epsilon g$$

- This line adds **momentum** to the update.
- αv carries information from previous updates (like inertia).
- $-\epsilon g$ is the new gradient update.
- Combined, this builds up speed in consistent directions and reduces the effect of noise.

4. Parameter Update:

$$\theta \leftarrow \theta + v$$

- Instead of directly using the gradient, we update the parameters using the velocity vector v .

Parameter Initialization Strategies

When you train a neural network, we need to assign initial values to the weights (and sometimes biases). This process is called **parameter initialization**.

When training deep neural networks, **how you initialize the weights** can have a **huge impact** on:

- Convergence speed
- Gradient flow (avoiding vanishing/exploding gradients)
- The quality of the final solution

Improper initialization may:

- Cause the network to **converge very slowly**
- Lead to **dead neurons** (especially with ReLU activations)
- Make gradients either explode or vanish, especially in deep networks
- Get stuck (bad local minimum)
- Diverge (exploding gradients)
- Learn slowly or not at all

A good initialization:

- Breaks symmetry between neurons (each neuron should learn different things)
- Avoids vanishing gradients (very small gradients, causing no learning)
- Avoids exploding gradients (very large gradients, causing instability)
- Speeds up learning
- Sets up the model to generalize better

Basic Techniques

1. Random Initialization

- Weights are randomly sampled from a **uniform** or **normal distribution**.
- The scale (spread) of this randomness is very important.
 - **Too small** → weak signals, vanishing gradients
 - **Too large** → exploding activations or gradients

2. Symmetry Breaking

- If all neurons in a layer start with the same weights, they learn the same features.
- Random initialization ensures each neuron follows a different trajectory during training.

3. Scaling the Initialization

- To maintain signal flow across layers, the weights are often scaled based on:
- **Number of inputs (fan-in)**
- **Number of outputs (fan-out)**

Smart Initialization Strategies

Xavier (Glorot) Initialization

- Best for **sigmoid/tanh** activations.
- Balances the flow of information **forward and backward**.
- Weights are drawn from:

$$W \sim U \left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}} \right)$$

He Initialization

- Best for **ReLU** activation.
- Keeps signal strength high even when half of ReLU units are inactive.
- Weights are drawn from:

$$W \sim \mathcal{N}(0, \sqrt{\frac{2}{n_{in}}})$$

Orthogonal Initialization

- Weights are initialized as orthogonal matrices (like perpendicular vectors).
- Keeps signal stable across layers (used especially in RNNs).

Sparse Initialization

- Only a few weights per neuron are non-zero.
- Helps large networks avoid overfitting early on.

Bias Initialization

- Usually set to **0**.
- Sometimes a small positive constant (e.g., 0.1 for ReLU) to prevent units from "dying".
- For output layers: set biases to match the expected distribution (e.g., logit of class probabilities).

Learning-Based Initialization

Instead of random values, sometimes we initialize using:

- Pretrained models (e.g., pretraining on ImageNet)
- Unsupervised pretraining (like autoencoders)
- Transfer learning (weights from one task reused in another)

Problems Addressed by Good Initialization

- **Vanishing gradients:** When gradients shrink as they propagate backward, halting learning.
- **Exploding gradients:** When gradients grow exponentially, destabilizing learning.
- Proper scaling of weights prevents both problems by maintaining the variance of the forward activations and backward gradients across layers.

Algorithms with Adaptive Learning Rates

Setting the right learning rate is critical in training deep networks. A too-small learning rate leads to slow learning; a too-large one causes the training to diverge.

The idea behind adaptive learning rate methods is:

- Use different learning rates for different parameters.
- Automatically adjust those learning rates during training.

Early Solution: Delta-Bar-Delta Algorithm

Introduced by Jacobs (1988), it was one of the first attempts to adapt the learning rate per parameter.

main Idea:

- If a parameter's gradient keeps the same sign (e.g., always negative), it means we're on a consistent slope → increase learning rate.
- If the gradient changes sign (e.g., from negative to positive), it might mean we overshoot the minimum → decrease learning rate.

Limitation:

- It works well only with full-batch training (i.e., using the entire dataset at once), which is not practical for modern large-scale deep learning.

Modern Adaptive Methods

As training shifted to **mini-batches** (small batches of data), newer algorithms were needed to:

- Adapt learning rates for each parameter
- Work with mini-batch gradients
- Handle noisy gradient estimates efficiently

This leads to modern adaptive optimizers like:

- **AdaGrad**
- **RMSProp**
- **Adam**

These optimizers learn how to adjust the learning rate **automatically** for each parameter **during training**.

AdaGrad

AdaGrad stands for **Adaptive Gradient Algorithm**.

It's an optimization algorithm that **adapts the learning rate for each parameter** individually during training.

Instead of using a single learning rate for all weights, AdaGrad:

Scales down the learning rate **proportionally to the historical sum of squared gradients** for each parameter.

This means: parameters that learn **quickly** (large gradients) will **slow down** over time. And parameters that learn **slowly** (small gradients) will retain **higher learning rates**.

So:

Big gradients → slower updates

Small gradients → faster updates

This helps the optimizer to:

- Take **larger steps** in **flatter** directions of the loss surface.
- Take **smaller steps** in **steep** directions.

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ

Require: Initial parameter θ

Require: Small constant δ , perhaps 10^{-7} , for numerical stability

Initialize gradient accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$

Compute update: $\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$. (Division and square root applied element-wise)

Apply update: $\theta \leftarrow \theta + \Delta\theta$

end while

- L is the loss function (e.g., MSE or cross-entropy).
- $\mathbf{g}$ is the average gradient of the loss w.r.t. θ on the minibatch.

Accumulate squared gradients:

$$\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$$

- $\odot$ means element-wise multiplication.
- This stores how much each parameter has been changing.

Compute the adaptive update:

$$\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$$

- Each parameter has a **custom learning rate** scaled by $1 / \sqrt{r_i}$.
- Parameters with **large accumulated gradients** get **smaller updates**.

RMSProp

RMSProp (Root Mean Square Propagation) is an adaptive learning rate optimization algorithm - just like AdaGrad — but it fixes one major problem with AdaGrad, i.e. It doesn't let the learning rate decay too much.

Problem with AdaGrad:

- AdaGrad accumulates all past squared gradients.
- Over time, this makes the denominator in its update rule grow very large, causing the effective learning rate to shrink toward zero.

Sol: Instead of accumulating the entire history, RMSProp uses an exponentially decaying average of the squared gradients.

RMSProp (Root Mean Square Propagation) is an adaptive learning rate algorithm that:

- Adjusts the learning rate individually for each parameter.
- Uses a moving average of squared gradients to prevent the learning rate from shrinking too much (a problem in AdaGrad).

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ , decay rate ρ .

Require: Initial parameter θ

Require: Small constant δ , usually 10^{-6} , used to stabilize division by small numbers.

Initialize accumulation variables $\mathbf{r} = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$

 Compute parameter update: $\Delta \theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

Update squared gradient average:

$$\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$$

Requirements

- ϵ : Global learning rate.
- ρ (rho): Decay rate, controlling how much past gradients are remembered.
- θ : Initial model parameters.
- δ : Small constant (e.g. $1e-6$) to prevent division by zero.
- $\mathbf{r}$ holds the **moving average of squared gradients**.

- This is an **exponentially weighted moving average** of squared gradients.
- ρ (typically around 0.9) controls how fast past gradients decay.
- It smoothens the contribution of past gradients compared to AdaGrad's accumulation.

Compute parameter update:

$$\Delta \theta \leftarrow -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$$

- Each parameter's update is scaled by the **inverse RMS** of recent gradients.
- Makes large updates in directions with **small variance**, and small updates where gradient magnitudes are large.

Adam Optimizer

Adam stands for **Adaptive Moment Estimation**.

It is one of the most widely used optimizers in deep learning today.

It is a combination of:

- **Momentum** (tracks past gradients to smooth updates)
- **RMSProp** (adapts learning rates based on recent squared gradients)

Adam combines both stability and adaptiveness.

1. Tracks first moment (mean of gradients) → like momentum

- Helps keep a consistent update direction

2. Tracks second moment (variance of gradients) → like RMSProp

- Helps scale learning rate properly for each parameter

3. Bias Correction

- Since both moment estimates start at zero, Adam includes **correction terms** early in training to prevent bias in those estimates.

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization. (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with
 corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

 Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

 Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

 Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

 Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

 Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

Choosing the Right Optimizer: Which One Is Best?

- **Deep learning problems are diverse**, and different tasks respond better to different optimizers.
- There's **no single algorithm** that consistently outperforms all others across all tasks.

Optimizer	Comments
SGD	Simple and widely used; can be slow without momentum
SGD + Momentum	Accelerates convergence and smooths updates
RMSProp	Adapts learning rates; works well for RNNs and deep nets
RMSProp + Momentum	Combines benefits of both; very stable
AdaDelta	Robust variant of AdaGrad; less commonly used today
Adam	Most popular; combines momentum + RMSProp; good default

Factor	Recommendation
You want simplicity and control	Try SGD or SGD + Momentum
You want faster convergence	Use Adam (great default choice)
You work with RNNs or unstable tasks	Use RMSPprop or RMSPprop + Momentum
You're new to tuning optimizers	Start with Adam